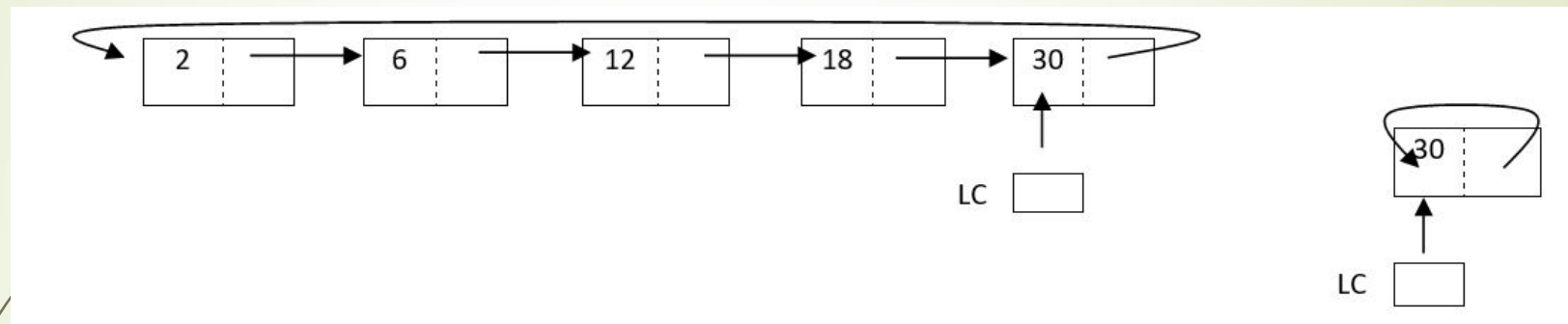


Listas Circulares

En esta estructura, el ultimo nodo está vinculado con el primero, permitiendo acceder a cualquier nodo independientemente de dónde se parta.

Ejemplo de dos listas con 5 y 1 nodos respectivamente.



LC almacena la dirección del último nodo, éste en su campo siguiente contiene la dirección del primer nodo de la lista ordenada.

Mantener la dirección del último, en lugar del primero, permite insertar al final de la lista sin tener que recorrerla. Para insertar al comienzo se recurre al último nodo y se modifica su campo siguiente con la dirección del nodo nuevo.

Listas Circulares - Tarea

Ej 1.- Mostrar el contenido de una lista circular.

```
typedef struct nodo {  
    int dato;  
    struct nodo *sig;} nodo;  
typedef struct nodo * TListaC;
```

Listas Circulares - Ejercicio

Ej 2.- Se tiene una lista circular ordenada que almacena palabras y frecuencia de aparición de cada una de ellas.

a) Insertar una palabra en la lista

b) Eliminar una palabra de la lista

```
typedef struct nodo {  
    int cant;  
    char palabra [15];  
    struct nodo *sig;} nodo;  
typedef struct nodo * TListaC;
```



```
a) void insertaPal ( TListaC *LC, char *cad){
    TListaC nuevo, aux, ant;

    nuevo = (TListaC) malloc (sizeof (nodo));
    strcpy(nuevo->palabra,cad);
    nuevo->cant= 1;
    if ((*LC)== NULL)    {
        (*LC) = nuevo;
        nuevo->sig = nuevo;
    }
    else
        if (strcmp(cad, (*LC)->palabra)>0) {
            nuevo->sig = (*LC)->sig;
            (*LC)->sig = nuevo;
            (*LC) = nuevo ;
        }
        else {
            ant = *LC;
            aux = (*LC)->sig;
            while (strcmp(cad, aux->palabra) >0) {
                ant = aux;
                aux = aux->sig;
            }
            if (strcmp(cad,aux->palabra)==0) {
                free(nuevo);
                aux->cant ++;
            }
            else {
                ant-> sig = nuevo;
                nuevo->sig = aux;
            }
        }
    }
}
```